CMSC 218 Review

Dylan Tang

May 2026

1 Pandas

- **Group by**: Collapses rows that share the same attribute into one row, and performs an aggregate operation on the collapsed rows.
- Ex. `df.groupby(["col"]).mean()`
- To group with multiple columns, use `agg()`:

```
df.agg(col1_mean = ("col1", "mean"), col2_median = ("col2", "median"))
```

- **Merge**: Merge by a common column
- Ex. `merged = pd.merge(df1, df2, on="id", how="inner")`
- `how`:
  - `inner` - merge takes intersection of both `df1["id"]`, `df2["id"]`
  - `outer` - merge takes union of `df1["id"]`, `df2["id"]`
  - `left` - merge uses `df1["id"]` as index
  - `right` - merge uses `df2["id"]` as index
- `outer`, `left`, `right` can produce NaN.

Example (Group by Departments)

```
df.groupby(["Department"])["Salary"].mean()
```

- The above line can be read as: group by department, and take the mean of each department's salary.
- More generally, groupby takes on the following form:

```
df.groupby([group_var])[value_var].agg_func()
```

Example (Pandas Merge with Duplicate Keys)

```
df1 = pd.DataFrame({"id": [1, 1, 2, 3], "dept": ["sales", "VP", "tech", "hr"]})
df2 = pd.DataFrame({"id": [1, 2, 2, 3], "hrs": [10, 20, 30, 15]})
df1.merge(df2)
```

The above merged df has the following form:

id	dept	hrs
1	sales	10
1	VP	10
2	tech	20
2	tech	30
3	hr	15

2 Sampling

- Given an RV X, $Var(X) = E[(X - E[X])^2] = E[X^2] - E[X]^2$
- **68-95-99.7 rule** - 68% of samples fall within 1 SD, 95% of samples fall within 2 SD, 99.7% of samples fall within 3 SD
- **Stratified samples** - point estimate is a weighted average of sub populations
  - $\bar{X} = w_1\bar{X}_1 + \ldots + w_n\bar{X}_n$
  - $\text{Var}(\bar{X}) = w_1^2\text{Var}(\bar{X}_1) + \ldots + w_n^2\text{Var}(\bar{X}_n)$
- Ensures representativeness of all populations according to preset weights
- **95% CI for proportions**:

$$C = \hat{p} \pm 1.96 \cdot \text{SE}(\hat{p})$$
$$\leq \hat{p} \pm 1.96\sqrt{\frac{0.5(0.5)}{n}}$$

3 Linear Model

- $Y = mX + b + \varepsilon$
- $\mathbb{E}[Y] = m\mathbb{E}[X] + b$
- $\text{Var}(Y) = m^2\text{Var}(X) + \text{Var}(\varepsilon)$
- **Signal-Noise Ratio**:

$$\text{SNR} = \frac{m^2\text{Var}(X)}{\text{Var}(\varepsilon)}$$

- $\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$
- Affected by scaling: $\text{Cov}(aX, Y) = a \cdot \text{Cov}(X, Y)$
- Measures the **scale** of association between X and Y
- **Correlation**:

$$\rho = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}}$$

- Not affected by scaling $X \sim Y$
- Measures the **strength**
- **1-D OLS**:

$$\hat{m} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} \Rightarrow \text{if } X, Y \text{ standardized}, \quad \hat{m} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} = \text{Cov}(X, Y)$$

$$\hat{b} = \mathbb{E}[Y] - \hat{m}\mathbb{E}[X]$$
$$\rho = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}} = \text{Cov}(X, Y)$$
$$\Rightarrow \hat{m} = \rho$$

4 Bias-Variance Decomposition

$$\mathbb{E}[(\hat{y}-y)^2] = \underbrace{(\mathbb{E}[\hat{m}]-m)^2}_{\text{Bias}^2(\hat{m})} + \underbrace{(\mathbb{E}[\hat{b}]-b)^2}_{\text{Bias}^2(\hat{b})} + \underbrace{\text{Var}(\hat{m})+\text{Var}(\hat{b})}_{\text{Variance}} + \underbrace{\text{Var}(\varepsilon)}_{\text{Error}}$$

5 Multivariate OLS

- $y = X\beta + \varepsilon, \quad X \in \mathbb{R}^{n\times p}$, $\beta \in \mathbb{R}^{p\times 1}$, $\varepsilon \in \mathbb{R}^{n\times p}$, $y \in \mathbb{R}^{n\times p}$
- $\arg\min_\beta |Y - X\beta|^2 \Rightarrow \hat{\beta} = (X^TX)^{-1}X^Ty \Rightarrow \hat{y} = X\hat{\beta} = X(X^TX)^{-1}X^Ty$
- $\beta_j \neq \frac{\text{Cov}(X_j, y_j)}{\text{Var}(X_j)}$, so $\hat{m} \neq \rho$ after standardizing X, y.
- A large β_j only indicates that one predictor is important in context of other predictors.
- $R^2 = 1 - \frac{\sum(\hat{y}-y)^2}{\sum(\bar{y}-y)^2} = 1 - \frac{\text{SSE}}{\text{SST}}$
- Measures how much better we do than the baseline model, $\bar{y}$.

5.1 Ridge Regression

Smooth predictions by adding L2 regularizer:

$$\arg\min_\beta |Y - X\beta|^2 + \alpha|\beta|^2 \Rightarrow \hat{\beta} = (X^TX + \alpha I)^{-1}X^Ty \Rightarrow \hat{y} = (X^TX + \alpha I)^{-1}y$$

6 Building a Regression Predictor

- **Inference model**: Given (x_i, y) pairs, produce $f(\theta)$.
- **Prediction model**: Given X, y as inputs, organizes the inputs in (X, y) pairs.

Typical regression pipeline:

```
X = df[...]
y = df[...]
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
r2_score(y_test, y_pred)
```

7 Generalization

- T_e - expected testing loss
- T_r - expected training loss
- $T_e - T_r = G$ (generalization gap)

$$\underbrace{(T_e - T_r)}_{G} + T_r = T_e$$

- A complex model is well-fit to the training data, and as such it is less generalizable - $G \uparrow$, $T_r \downarrow$.
- A simple model likely can be well generalized, but has high training error - $G \downarrow$, $T_r \uparrow$.

- A good model jointly minimizes G, T_r.

This tradeoff is also seen in the Bias-Variance Decomposition for squared loss:

$$\mathbb{E}[\ell(y_{\text{out}}, y_{\text{true}})] = \mathbb{E}[\ell(\hat{f}(x), y_{\text{true}})]$$

$$\mathbb{E}[(y_{\text{true}} - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - y_{\text{true}})^2}_{\text{Bias}} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}$$

- A simple model has less variance, but more bias
- A complex model has low bias, high variance.
- OLS jointly optimizes over bias and variance by minimizing squared loss.

8 Autoregression

- **Autoregression**: Using past values of y to predict future values of y.
- Any model that predicts an outcome based on time can may be subject to autoregression.
- **AR(1) Model**:

$$y_t = \underbrace{\beta_{\text{lag}} y_{t-1}}_{\text{AR(1) param}} + \underbrace{X\beta}_{\text{other covariates}}$$

Typical AR(1) Pipeline:

```
df["y_{t-1}"] = df["y"].shift(1)
df.dropna(how="any")

X = df[...]
y = df[...]

tscv = TimeSeriesSplit()
for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
r2_score(y_test, y_pred)
```

- **Longitudinal model** - uses $y_{t-k}, \ldots, y_{t-1}$ to predict y_t
- **Cross-sectional model** - uses covariates X to predict y_t

9 Logistic Regression

Given (X_i, y_i), define the probability y_i belongs to class 1 as

$$\hat{p}_i = \frac{1}{1 + e^{-w^T x_i + b}}$$

Then, minimizes cross-entropy loss:

$$\ell(y, \hat{p}) = \sum_i [y_i \log(\hat{p}_i) + (1 - y_i)(1 - \hat{p}_i)]$$

- By converting X_is to $\hat{p}_i$s, reduces influence of x_is that are outliers/high leverage points

10 Classification

- **Binary Classification**: Train model on weights w, bias b s.t. the decision boundary $w^T x+b$ maximizes correct predictions.
- **Confusion Matrix**:

	Actual	
Predicted	T	N
T	TP	FP
N	FN	TN

10.1 Multiclass Classification

- Instead of $y_i \in \{0, 1\}$, now $y_i \in \mathcal{L}$, where $\mathcal{L} = \{1, \ldots, k\}$.
- Models for multiclass classification: Logistic Regression, NN/CNN with Cross Entropy loss

Metrics:

- **Accuracy**: $\dfrac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$
- **Precision**: $\dfrac{\text{TP}}{\text{TP} + \text{FP}} = \dfrac{\text{True positives}}{\#\text{ of predicted positives}}$
  - How accurate are the model's positive classifications
- **Recall**: $\dfrac{\text{TP}}{\text{TP} + \text{FN}} = \dfrac{\text{True positives}}{\#\text{ of actual positives}}$
  - What is detection rate for the true positives!

Example (Multiclass):

	Predicted		
Actual	1	2	3
1	n_{11}	n_{12}	n_{13}
2	n_{21}	n_{22}	n_{23}
3	n_{31}	n_{32}	n_{33}

$$\text{Precision of class 2} : \frac{n_{22}}{n_{12} + n_{22} + n_{32}}$$

$$\text{Recall of class 2} : \frac{n_{22}}{n_{21} + n_{22} + n_{23}}$$

11 Two Sample Z-Test

$$\begin{aligned} H_0 : \mu_A = \mu_B & \quad \text{There is no difference in the two populations.} \\ H_A : \mu_A \neq \mu_B & \quad \text{There is a difference in the two populations.} \end{aligned}$$

$$z^* = \frac{\bar{x}_A - \bar{x}_B - 0}{\frac{\sigma_A}{\sqrt{n_A}} + \frac{\sigma_B}{\sqrt{n_B}}}$$

If $P(z > z^*) < \alpha$, reject H_0 for H_A.

- P-value is the false discovery rate.

12 Efficiency vs. Robustness

- **Efficiency** - tries to use as much of the training data as possible to reach a conclusion
- **Robustness** - protects against bad assumptions, noise, or inaccurate data
- Regularization is an example of robustness. Ridge regression, for example imposes prior $w \sim N(0, \frac{1}{\lambda})$.
  - Trade off a small increase in bias for a large decrease in variance
- Other robustness methods:
  - Acquire more data to decrease leverage of an influential point
  - Convert data to percentiles or ranks (Low Rank Prior)
    - Can lose some of the data's meaningfulness, but improves robustness
  - Find a low-rank representation of data (PCA)

13 PCA

- Rank k dimension reduction given by
$$X_k = U_k \Sigma_k {V^T}_k$$
- Reduce high-dimensional data along a low-dimensional basis composed of principal components (PCs).
  - Each PC is in the direction of greatest variance in the data
- To find which basis is the result of PCA check:
  - Does the basis vector have stronger components in variance maximizing areas.
  - Does the basis keep positive/negative correlations as exhibited in the data?

14 Neural Networks

14.1 Forward Propagation

- Given input layer $X \in \mathbb{R}^{n \times p}$, forward propagation is defined by:
- At X (layer 0):
$$Z^{(0)} = W^{(0)} X + b^{(0)}, \quad A^{(0)} = \sigma(Z^{(0)})$$
- At hidden layer ℓ:
$$Z^{(\ell)} = W^{(\ell)} A^{(\ell-1)} + b^{(\ell)}, \quad A^{(\ell)} = \sigma(Z^{(\ell)})$$
- Then, $\hat{Y} = A^{(L)}$, and the loss function is given by $\ell(Y, \hat{Y})$ at layer L.

14.2 Back Propagation

$$\frac{\partial \ell}{\partial W^{(\ell)}} = \frac{\partial \ell}{\partial A^{(\ell)}} \cdot \frac{\partial A^{(\ell)}}{\partial Z^{(\ell)}} \cdot \frac{\partial Z^{(\ell)}}{\partial W^{(\ell)}}$$
$$= \left[\frac{\partial \ell}{\partial A^{(\ell)}} \odot \sigma'(Z^{(\ell)}) \right] (A^{(\ell-1)})^T$$

$$\frac{\partial \ell}{\partial A^{(\ell)}} = \frac{\partial}{\partial A^{(\ell)}} \ell(\hat{y}, y) = \frac{\partial}{\partial A^{(\ell)}} \ell(A^{(L)}, y) = \frac{\partial}{\partial A^{(\ell)}} \ell \left(\sigma(\underbrace{W^{(L-1)} A^{(L-1)} + b^{(L-1)}}_{Z^{(L)}}), y \right)$$

$$= \frac{\partial}{\partial A^{(\ell)}} \ell \left(\sigma(W^{(L-1)} \sigma(\underbrace{W^{(L-2)} A^{(L-2)} + b^{(L-2)}}_{Z^{(L-1)}}) + b^{(L-1)}), y \right)$$
$$\dots$$

Notes:

- $X_1, \dots, X_N$ should be standardized to prevent exploding/vanishing gradients
- One forward + backward pass is called an **epoch**
- With no activation function, we have that

$$\hat{y} = W^{(L-1)} W^{(L-2)} \dots W^{(1)} X + b^{(1)} + \dots + b^{(L-2)} + b^{(L-1)}$$

which is just a linear model.

```
from sklearn.preprocessing import StandardScalar
X_norm = StandardScalar().fit_transform(X)
```

14.3 Label Encoding, One-Hot Encoding

Label Encoding

```
from sklearn.preprocessing import LabelEncoder
X[label_encoded_var] = LabelEncoder().fit_transform(X[cat_col])
```

- Assigns each categorical variable outcome a numerical level.
- As such, label encoding assumes that there is an ordering in outcomes, when none may exist. This may make interpretation difficult, and may result in inflated coefficients.

One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder
encoded_X_cols = OneHotEncoder().fit_trnasform(X[cat_col])
```

- Assigns each categorical variable outcome a separate column (or indicator variable).
- Let l be an arbitrary outcome. If $\mathbf{1}_{\{x_i \in l\}}$, then for the column associated with l, x_i's value is 1.
- Unlike label encoding, one-hot encoding does not assume an ordering in outcomes.
- However, too many outcomes can lead to a significant increase in predictors, which can distill possible association and pull coefficients towards 0.

15 Convolution

- X is the input signal
- Y is the filter/kernel that slides along X

Ex. 1: $X = [a, b, c, d, e]$, $Y = [-1, 1]$

$$X * Y = [-a + b, -b + c, -c + d, -d + e]$$

All convolutions in this case are stride 1, no padding.

Ex. 2:

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$X * Y = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

A convolution is a fast way to disseminate information throughout the network.
Ex. 3: Given $X = [a \quad b \quad c \quad d \quad e]$, $f_1 = [f_{11} \quad f_{12}]$, $f_2 = [f_{21} \quad f_{22}]$

$$f_1 * X = [f_{11}a + f_{12}b, \quad f_{11}b + f_{12}c, \quad f_{11}c + f_{12}d, \quad f_{11}d + f_{12}e]$$

$$\begin{aligned} f_2 * f_1 * X = [&f_{21}(f_{11}a + f_{12}b) + f_{22}(f_{11}b + f_{12}c), \\ &f_{21}(f_{11}b + f_{12}c) + f_{22}(f_{11}c + f_{12}d), \\ &f_{21}(f_{11}c + f_{12}d) + f_{22}(f_{11}d + f_{12}e)] \end{aligned}$$

- In each component in $f_2 * f_1 * X$, there are three data points (a, b, c), (b, c, d), (c, d, e).
- But, the data points at the edges (a, e) are represented less than data points close to the middle.
- Each subsequent convolution reduces each dimension by 1
  - Given $X \in \mathbb{R}^{n \times m}$, $X * Y \in \mathbb{R}^{(n-1)\times(m-1)}$

16 Attention

- Given $X \in \mathbb{R}^d$, let $g \in \mathbb{R}^d$ be an attention filter of the same dimension.
- Then, each component of g_i determines how much attention is placed on the component X_{ij}.
-

$$Xg = \begin{bmatrix} -x_1- \\ \vdots \\ -x_n- \end{bmatrix} \begin{bmatrix} g_1 \\ \vdots \\ g_d \end{bmatrix} = \begin{bmatrix} x_{11} \cdot g_1 & x_{12} \cdot g_2 & \dots & x_{1d} \cdot g_d \\ \vdots & & & \vdots \\ x_{n1} \cdot g_1 & x_{n2} \cdot g_2 & & x_{nd} \cdot g_d \end{bmatrix}$$

17 Large Language Models

- LLMs are fundamentally next-token predictors
- Each word is assigned a unique token t
- Then, given tokens $t_1 \dots t_k$, predict t_{k+1}

LLMs are trained in the following way:

1. **Foundational Model Training** - learn weights from entire Internet
2. **Mid-training** - Refine weights on questions asked on the Internet, learn a small set of conversational data
3. **Post-training** - Using real people to finetune the model with Sophisticated Conversations

LLMs typically downsample user queries into a densely represented embedding vector. This embedding vector can then be compared to saved embeddings in a vector database, and can further

- **Encoder/Autoencoder** - reduce $X_1 \dots X_n$ to dense representation, then upsample the response.